Twelfth Implementation Iteration: Space Allocation

with Abstract Request Base Class

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this thirteenth implementation iteration is to improve the system architecture by consolidating all rule evaluation logic into a centralized rule engine and exposing it through a facade.

In previous iterations, the allocation service directly interacted with multiple rule classes. This created tight coupling and made it harder to extend or modify the rule set.

In this iteration, a `RuleEngine` and `RuleEngineFacade` are introduced. This isolates rule evaluation from the rest of the system and provides a single entry point for rule processing.

In this iteration, a new abstract class `Request` is introduced. This improves code reuse and enables polymorphic rule evaluation for the shared request properties.

2. Changes from Iteration 12

Compared with the twelfth iteration, the following changes were introduced:

- Introduced a new interface: `IRequestRule`
- Refactored rule classes (`CapacityRule`, `FeatureRule`, `StatusRule`, `LocationRule`) to implement `IRequestRule`
- Added `RuleEvaluationResult` to store rule outcomes and failure reasons
- Introduced `RuleEngine` to manage and apply all rules
- Introduced `RuleEngineFacade` to provide a single API for rule evaluation
- Refactored `AllocationService` to use only the facade instead of calling rules directly

3. Scenario

A user submits a request which is processed through the `RuleEngineFacade`. The facade delegates evaluation to the rule engine, which applies all rules sequentially.

If all rules pass, the request is approved and allocations are created. If any rule fails, the request is rejected and a failure reason is stored.

4. Modules and Their Tasks

A. Rule Engine Module

- Manage all request rules through a common interface
- Apply rules sequentially
- Return evaluation result and failure reason

B. Facade Module

- Provide a single API for rule evaluation
- Hide internal rule engine complexity from other modules

C. Allocation Service

- Delegate rule evaluation to the facade
- Handle allocation creation based on evaluation result

5. Iteration Comparison

Iter.	**Requests**	**Resources**	**Rules Added**
1–4	OTR Only	Class → Lab → Meet	Availability + Cap.
5–6	OTR + RR	Class + Lab + Meet	(Redesign)
7	OTR + RR	Class + Lab + Meet	+ Feature
8–9	OTR + RR	Class + Lab + Meet	+ Status
10–11	OTR + RR	Class + Lab + Meet	+ Location
12	Abstract Request	Class + Lab + Meet	Polymorphism
13	Abstract Reques	Class + Lab + Meet	**Rule Engine + Facade**

6. Updated Class Diagram

Figure 1 shows the updated system architecture including the rule engine and facade.

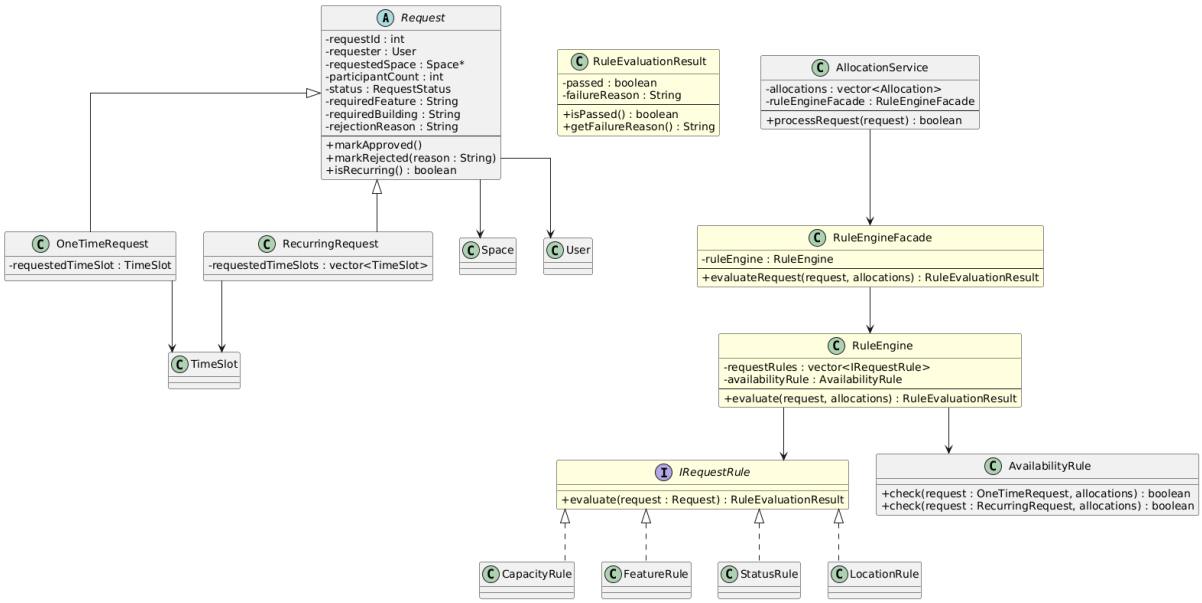

Figure 1: Updated Simplified Class Diagram

7. Updated Workflow Diagram

Figure 2 shows the simplified workflow where rule evaluation is handled by a centralized engine.

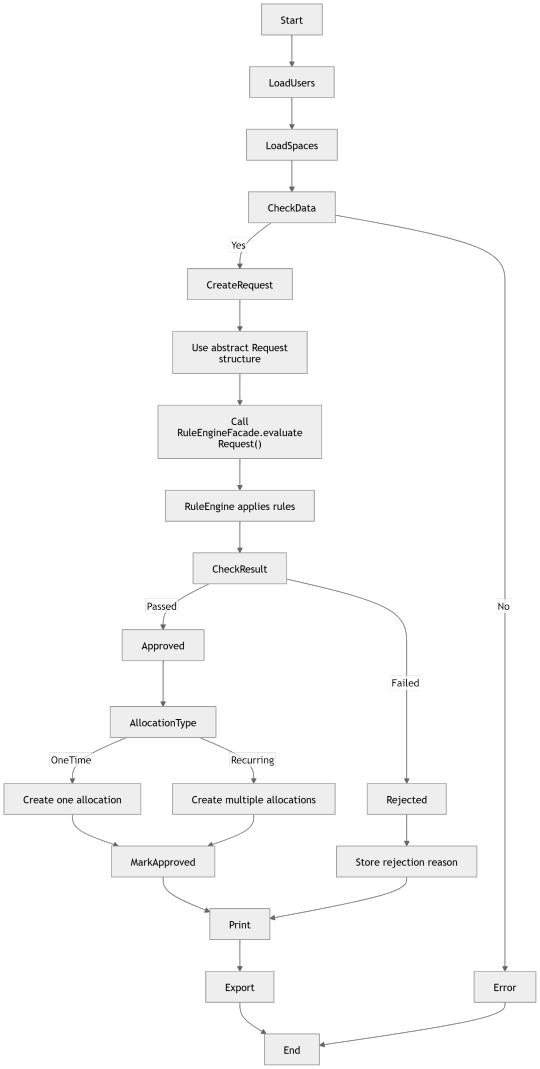

Figure 2: Updated Workflow Diagram

8. Prototype Implementation

The system was refactored to centralize rule evaluation. The allocation service now interacts only with the facade, improving modularity and extensibility.

9. Summary

This iteration significantly improves system flexibility and maintainability by isolating rule logic and introducing a clean architecture for rule evaluation.